

William Stallings

Data and Computer

Communications

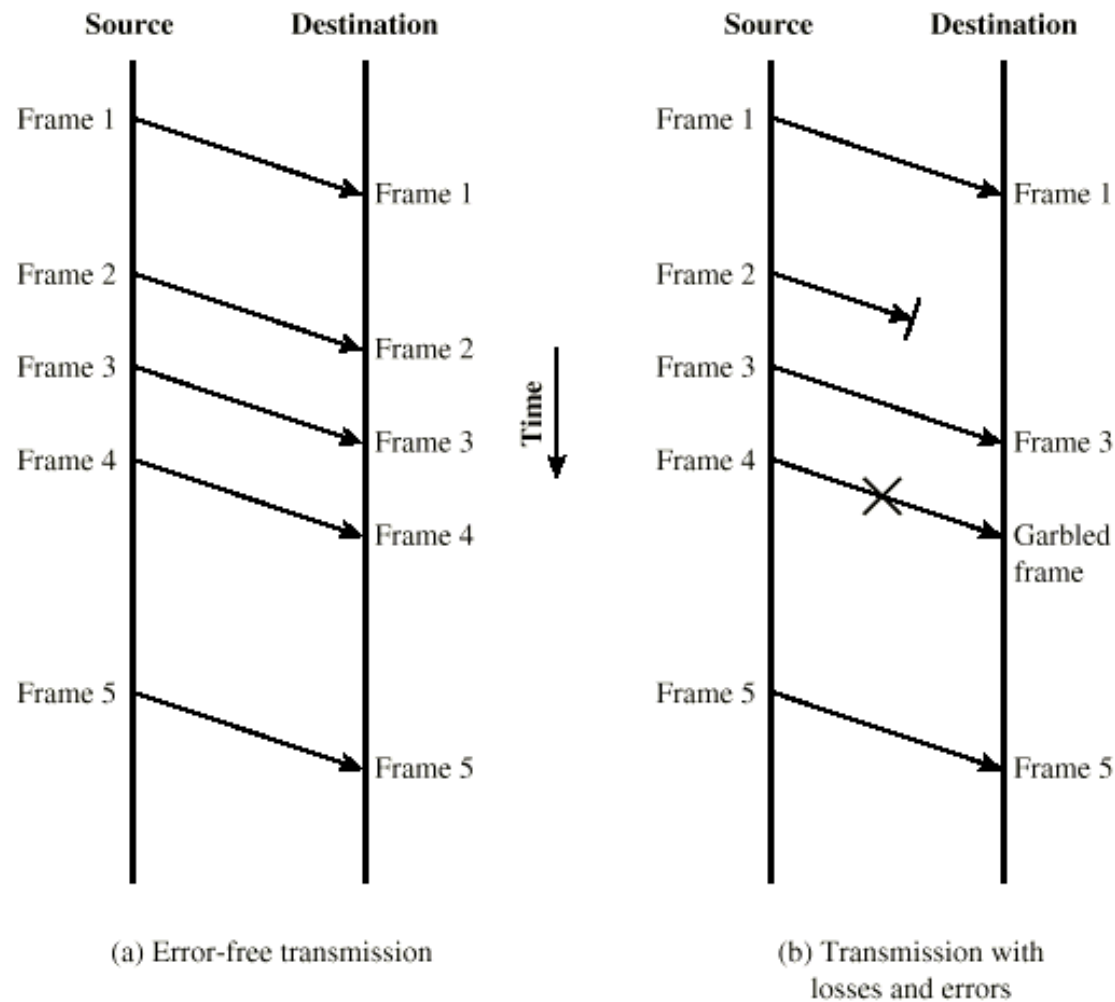
Chapter 7

Data Link Control

Flow Control

- Ensuring the sending entity does not overwhelm the receiving entity
 - Preventing buffer overflow
- Transmission time
 - Time taken to emit all bits into medium
- Propagation time
 - Time for a bit to traverse the link

Model of Frame Transmission



Stop and Wait

- Source transmits frame
- Destination receives frame and replies with acknowledgement
- Source waits for ACK before sending next frame
- Destination can stop flow by not send ACK
- Works well for a few large frames

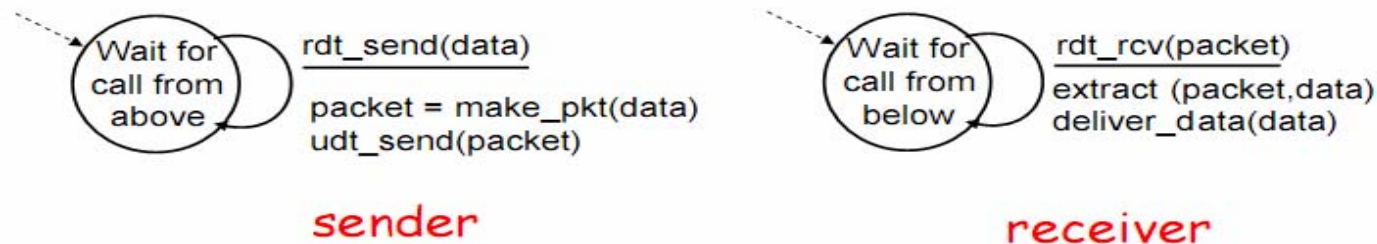
Fragmentation

- Large block of data may be split into small frames
 - Limited buffer size
 - Errors detected sooner (when whole frame received)
 - On error, retransmission of smaller frames is needed
 - Prevents one station occupying medium for long periods
- Stop and wait becomes inadequate

Protocolo de transferencia fiable sobre canal fiable

tdf1.0

- Canal perfectamente fiable
 - Sin errores a nivel de bit
 - Sin pérdidas de paquetes
- Diferentes máquinas de estado finito para el emisor y el receptor
 - Únicos estados en cada uno
 - Evento causante (sobre la línea)
 - Acción a realizar (bajo la línea)
 - Flechas indican la transición de los protocolos

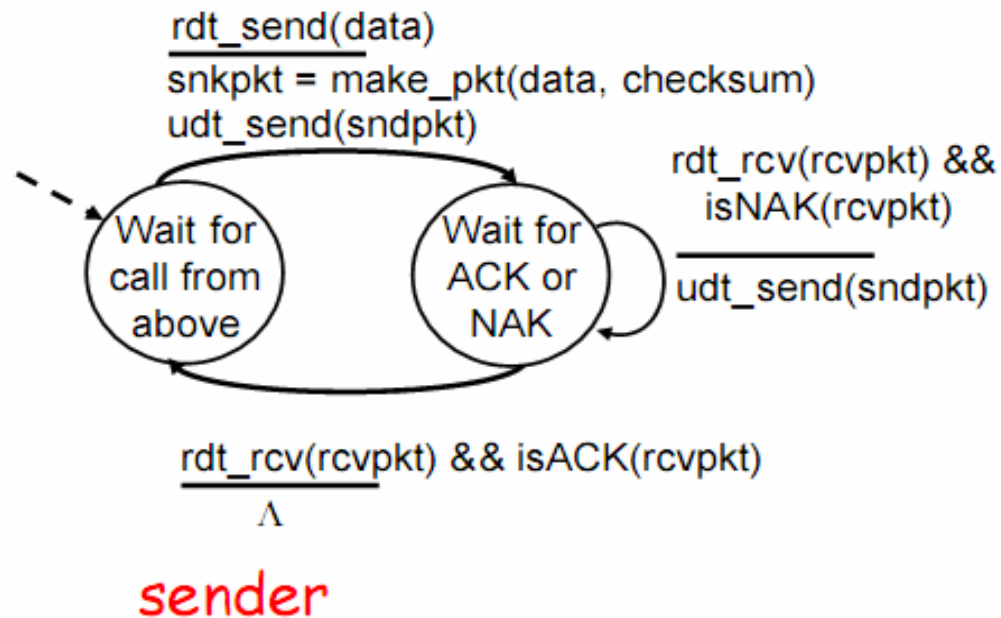


Protocolo de transferencia fiable sobre canal con errores de bit

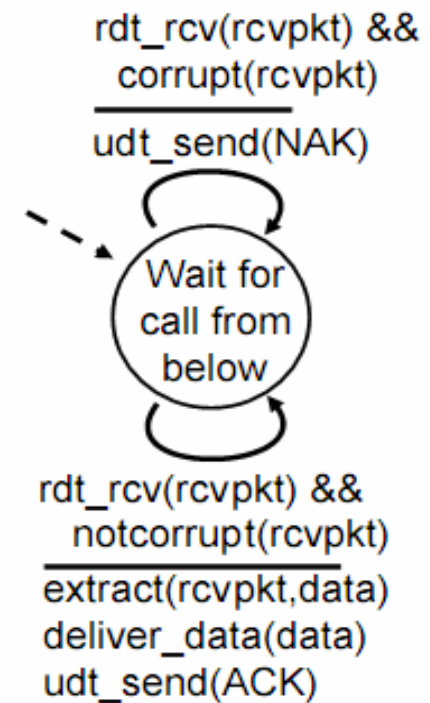
tfd2.0

- Canal mas realista donde la información se corrompe.
 - ❑ Checksum para detectar errores en la recepción de la información (bit).
- Como recuperar los errores ?
 - ❑ ACK (acknowledgements) : el receptor explícitamente avisa al emisor de la recepción conforme de la información.
 - ❑ NAK's (negative acknowledgements) : el receptor explícitamente avisa al emisor que los paquetes se recibieron con errores.
- tfd2.0 Nuevos mecanismos agregados a tfd1.0
 - ❑ Detección de errores
 - ❑ Retroalimentación del receptor al emisor

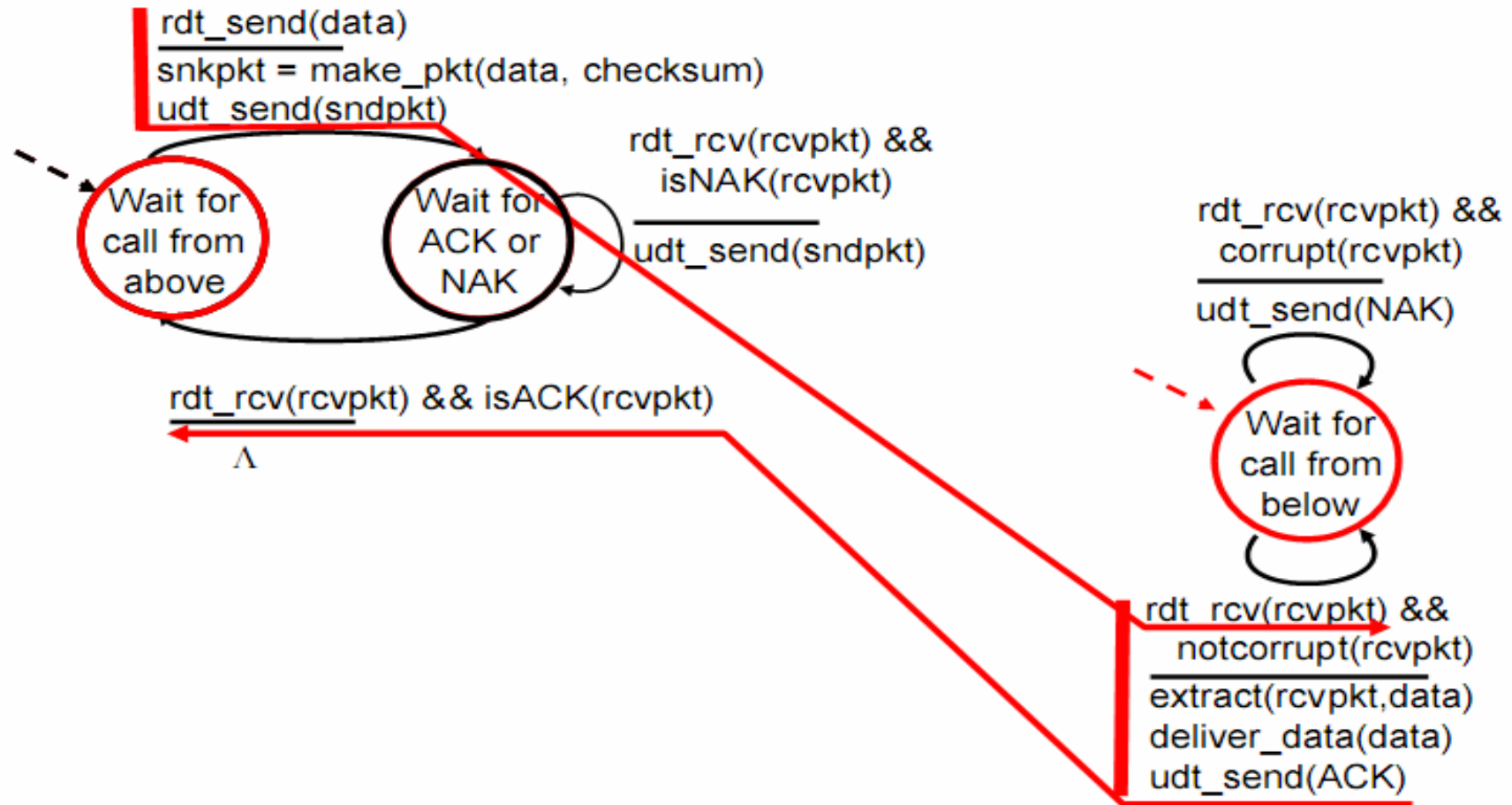
tfd2.0 - Especificación de FSM



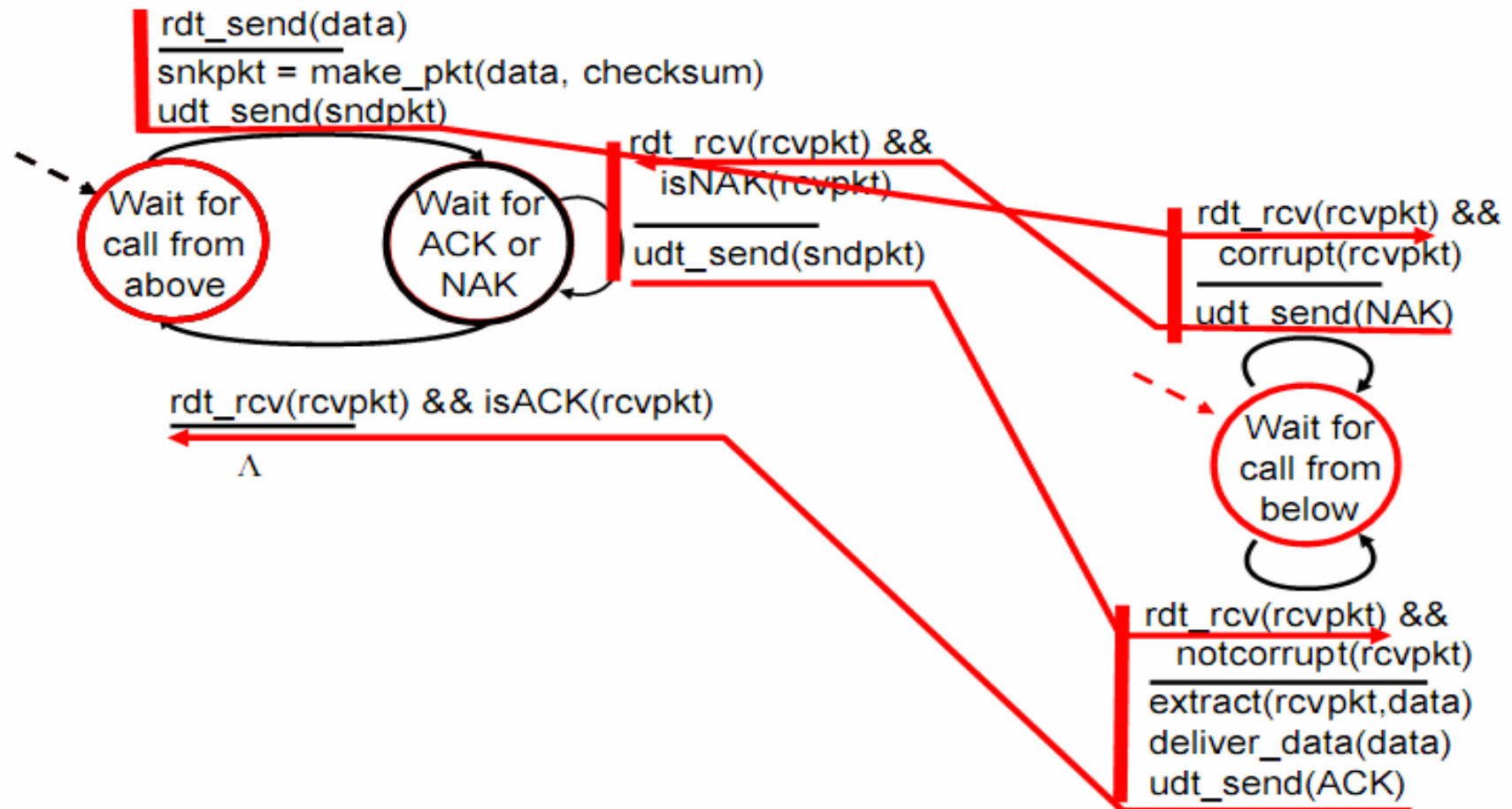
receiver



tfd2.0 - Operación sin errores



tfd2.0 - Operación con errores



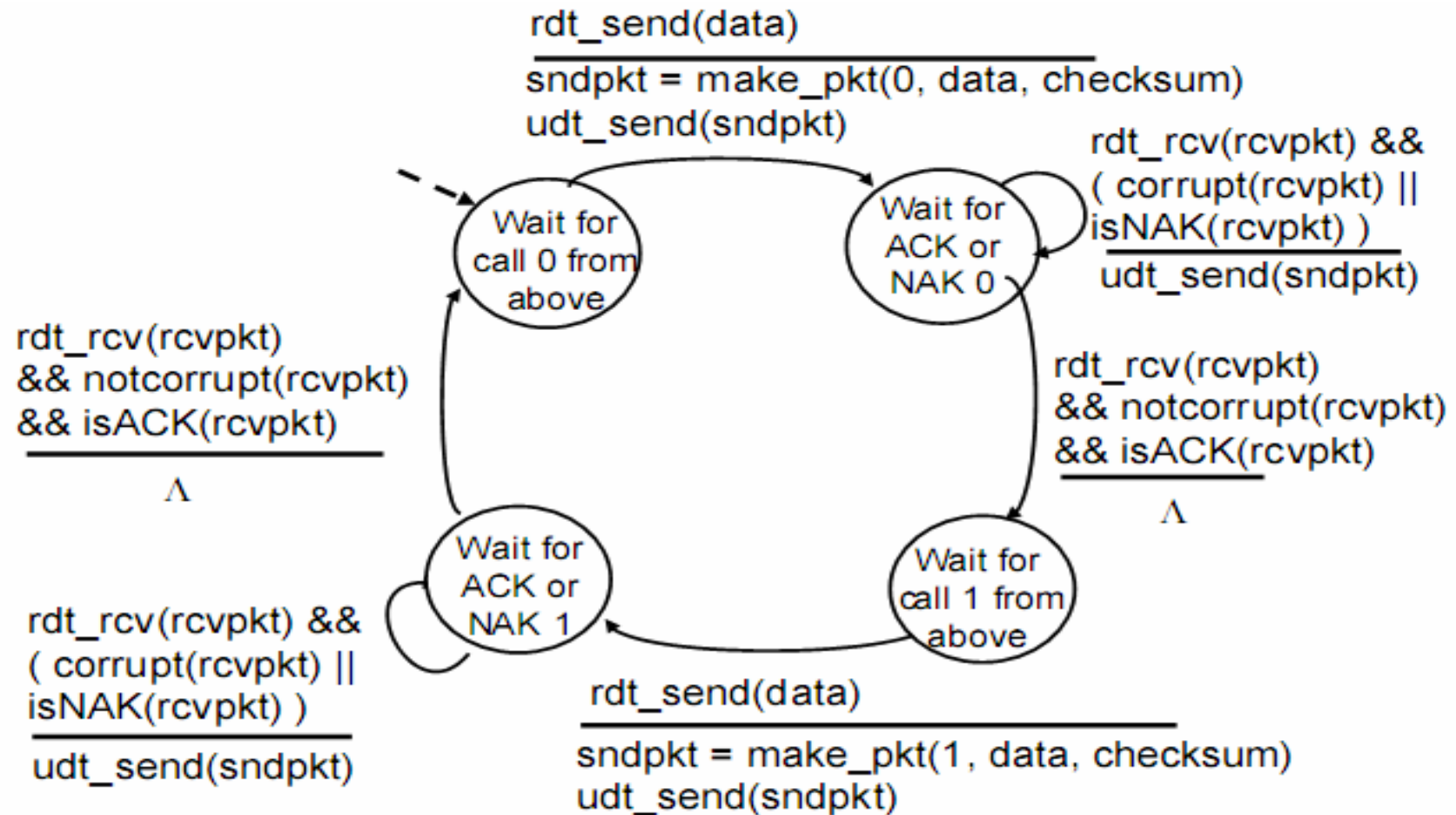
tfd2.0 – Error fatal !!!

- Que pasa si los paquetes de ACK/NAK están corruptos?
 - ❑ El emisor desconoce que paso con la recepción
 - ❑ Podría NO retransmitir
 - ❑ Podría transmitir en duplicado

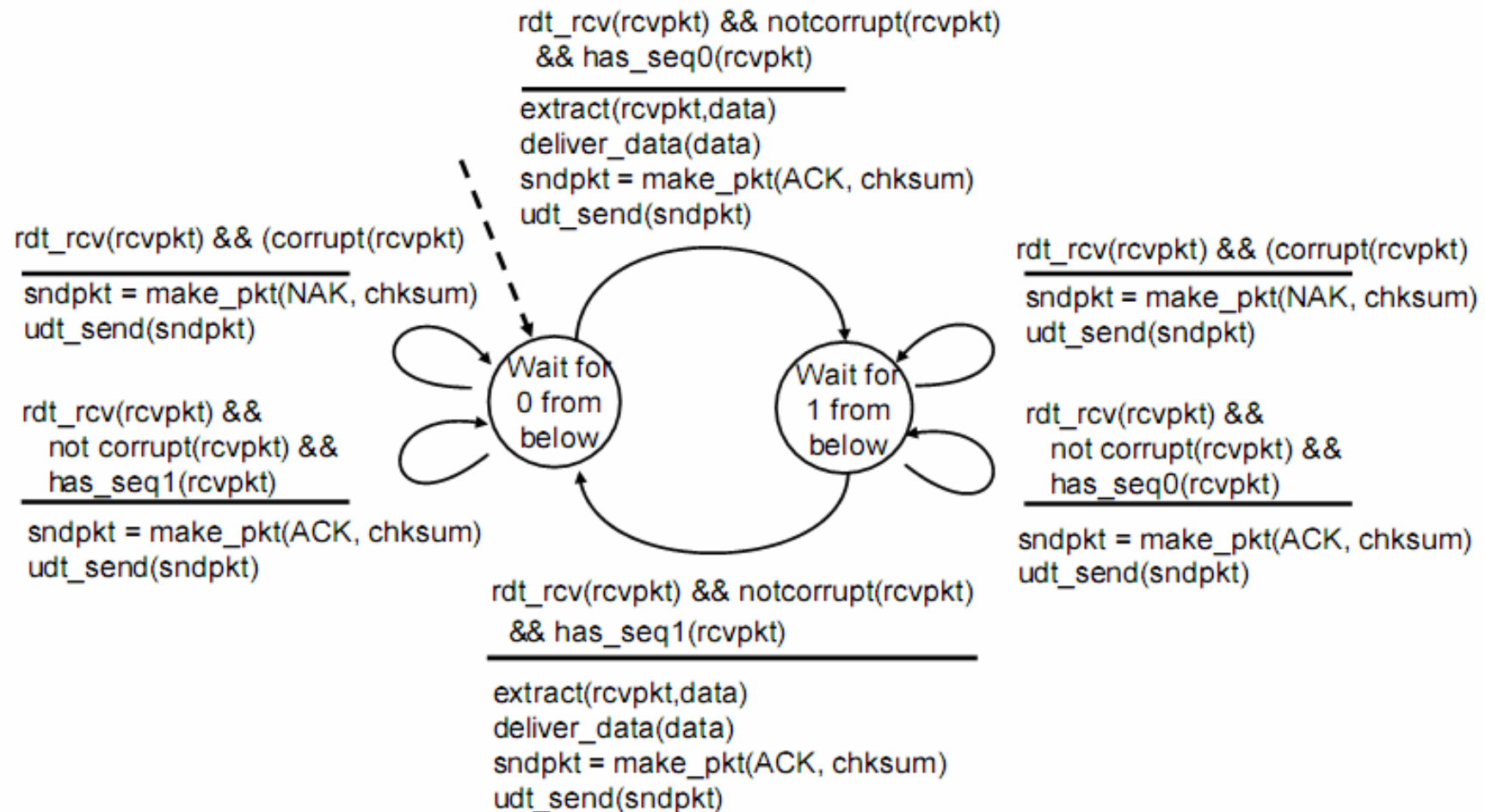
Protocolo de Parada y Espera

- Manejo de paquetes duplicados...
 - ❑ Agregar nuevos paquetes de control
 - ❑ Añadir mas bits de comprobación que permitan recuperar el error. Que pasa si se pierde el paquete?
 - ❑ Retransmitir paquetes si ACK/NAK están corruptos (paquetes duplicados)
 - ❑ Emisor agrega un número de secuencia a cada paquete que envía. (TCP lo hace).

tfd2.1 – Emisor maneja errores en ACK/NAK



tfd2.1 – receptor maneja errores en ACK/NCK



tfd2.1 – Características

■ Emisor

- ❑ Agrega numero de secuencia a cada paquete.
- ❑ Dos secuencias 0 y 1.
- ❑ Debe revisar si los ACK/NAK están corruptos.
- ❑ Debe recordar la secuencia del paquete enviado (0,1).

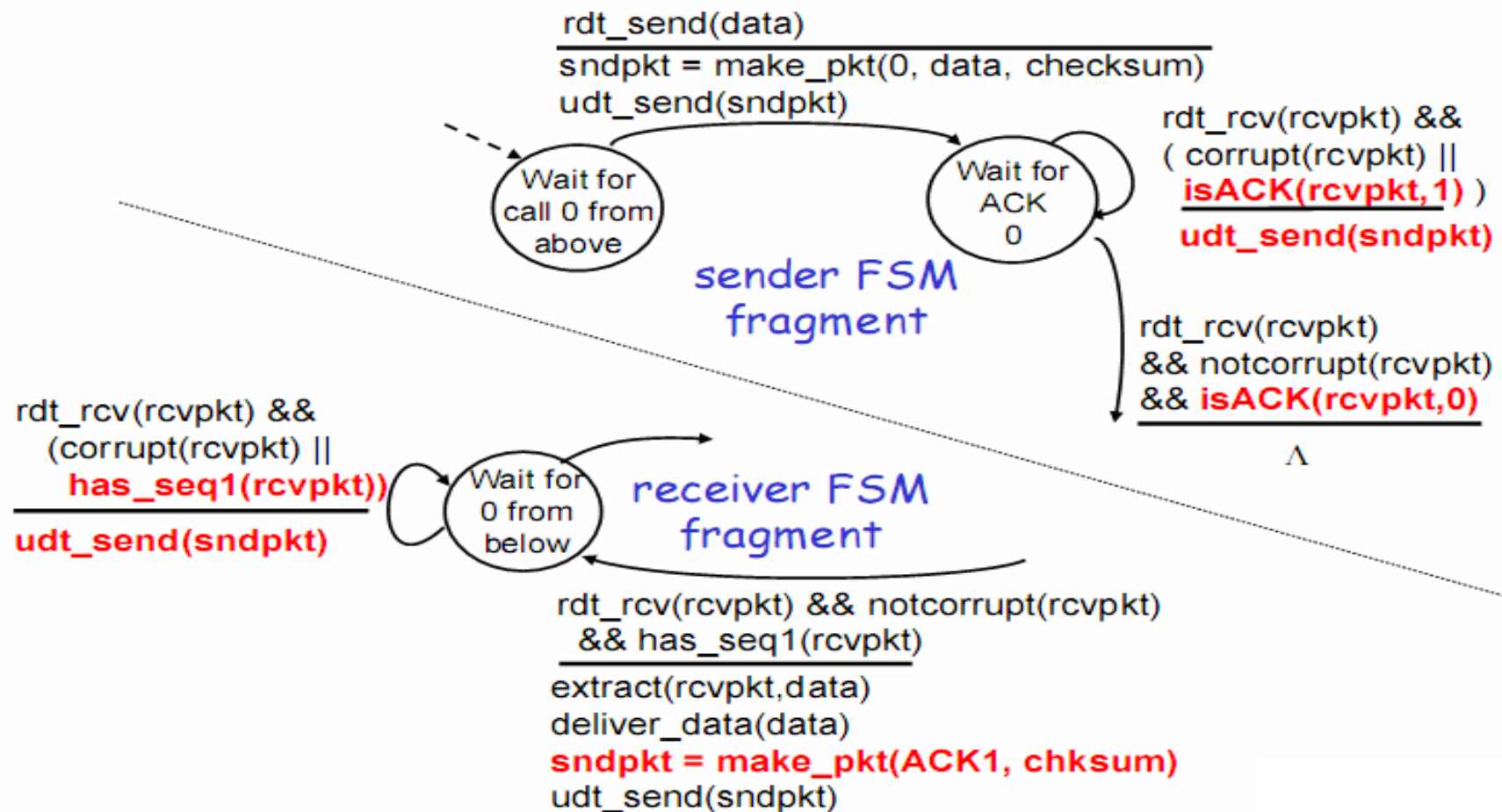
■ Receptor

- ❑ Revisar si el paquetes es duplicado. (0,1)
- ❑ No sabe si el emisor recibió bien el ultimo paquete de ACK/NAK.

tfd2.2 – un protocolo sin NAK

- Entrega las mismas funcionalidades de tfd2.1 pero no incorpora los paquetes de NAK.
- Receptor solo envía ACK para el ultimo paquete recibido OK.
 - Obligatoriamente el ACK debe incluir el número de secuencia del paquete recibido correctamente.
- ACK duplicados en el emisor provocan la misma acción que un NAK → retransmitir el paquete actual.

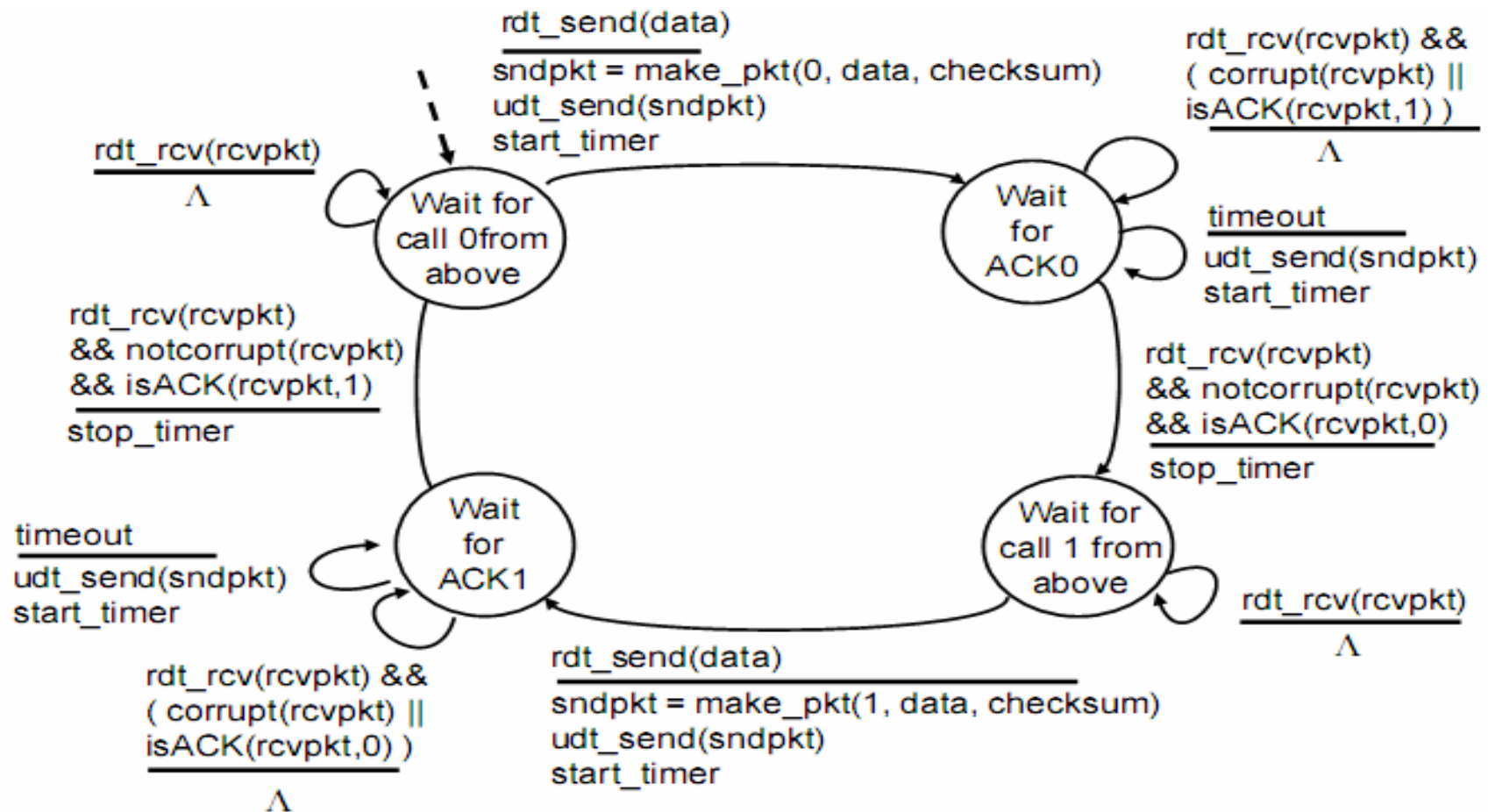
tfd2.2 – Fragmentos de FSM sin NAK



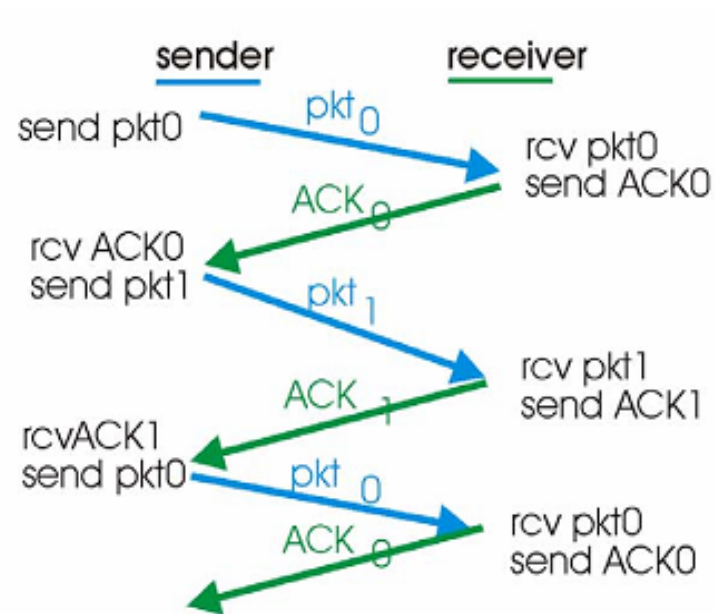
tfd3.0 – Canales con errores y pérdidas

- El canal no solo puede tener errores, también se pueden producir pérdidas de paquetes.
- Como detectar la pérdida y que hacer
 - ❑ Checksum
 - ❑ Número de secuencias
 - ❑ ACKs
 - ❑ Retransmisiones
- tfd2.2 No solucionan todo el problema, no detecta la pérdida
- Nuevo mecanismo en el protocolo de comunicación
 - ❑ Emisor debe esperar un tiempo t
 - ❑ Retransmitir si no hay ACK durante este tiempo t .
 - ❑ Si el PKT o el ACK sufren retraso (no pérdida) mayor a tiempo t
 - El paquete se retransmite, pero usa numero de secuencia. . .
 - Receptor debe especificar el numero del PKT al que corresponde al ACK.
 - ❑ Requiere un contador regresivo.

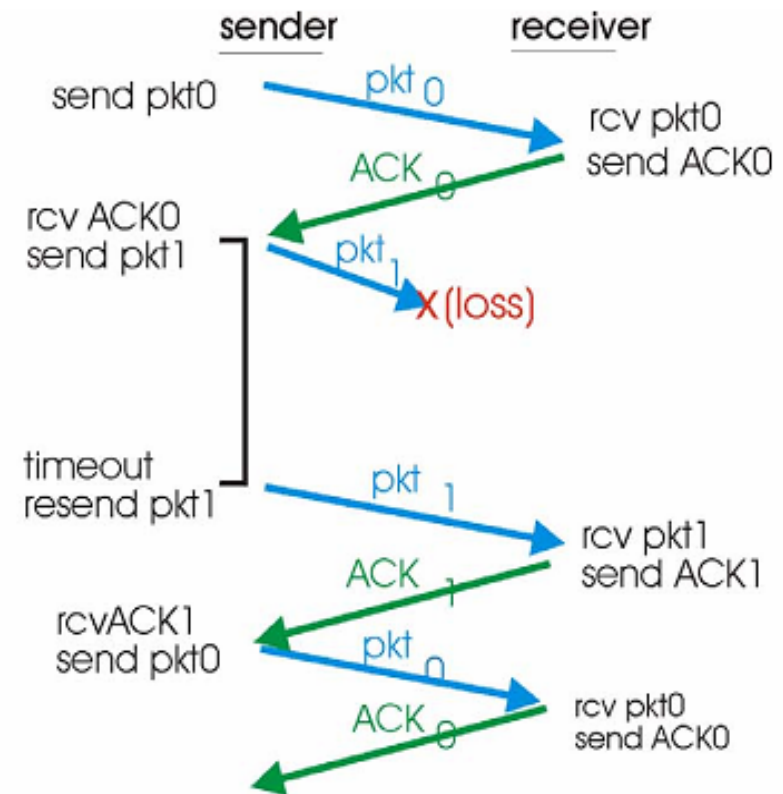
tfd3.0 – emisor



tfd3.0 – Funcionamiento

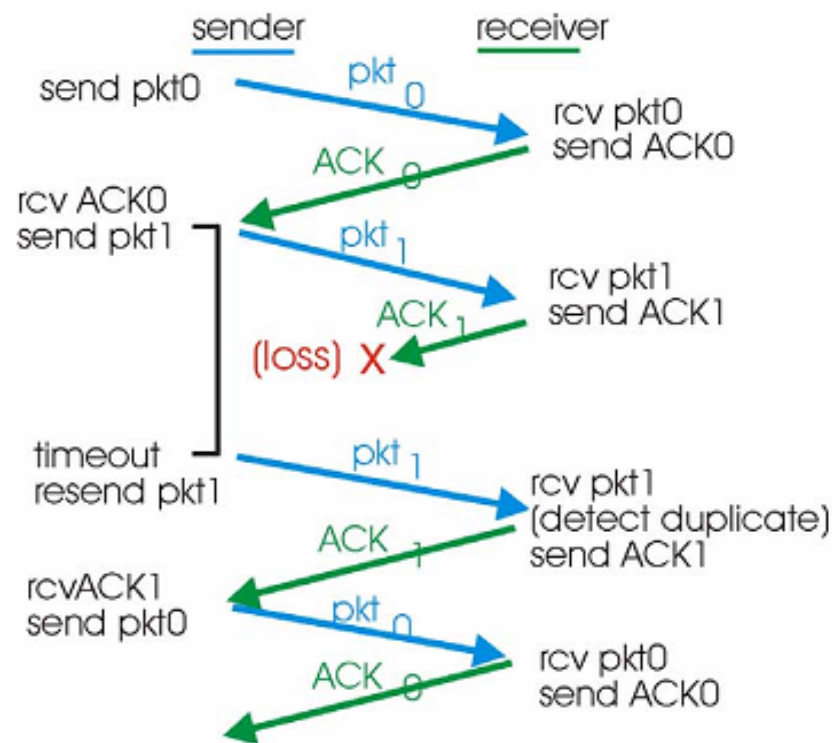


(a) operation with no loss

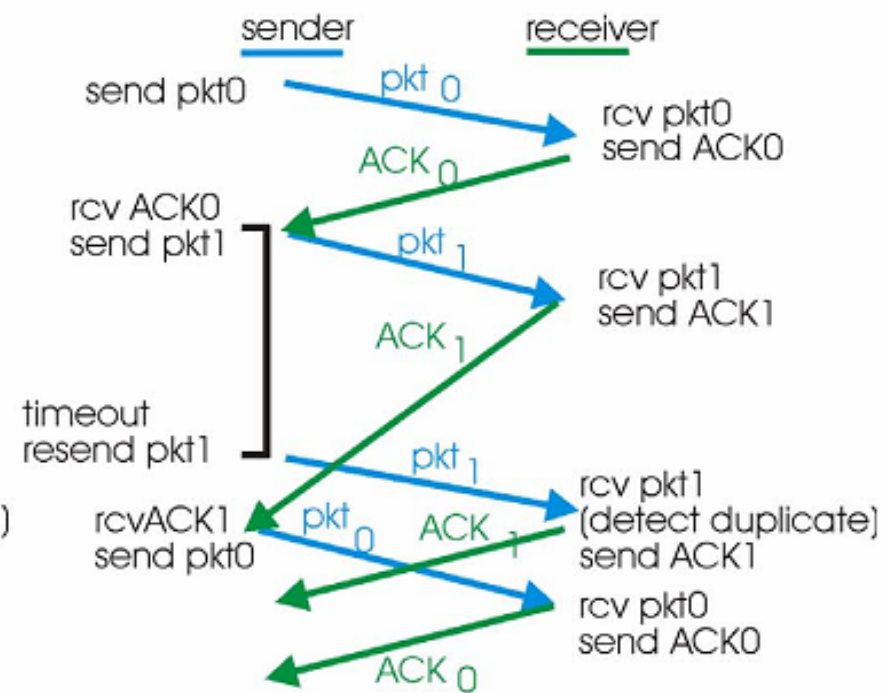


(b) lost packet

tfd3.0 – Funcionamiento



(c) lost ACK



(d) premature timeout

tfd3.0 – Características y Rendimiento

- tfd3.0 Funciona y se conoce como protocolo de bit alternante.
- Tfd3.0 Trabaja pero lo hace con un rendimiento fatal.
- Ejemplo:
 - ❑ Enlace de 1Gbps, ($R=10^9$ bps)
 - ❑ retardo E2E de 15ms ($RTT=30$ ms)
 - ❑ paquete de 1Kb: ($L=1000$ bytes = 8000 bits)

$$T_{\text{transmit}} = \frac{L \text{ (packet length in bits)}}{R \text{ (transmission rate, bps)}} = \frac{8\text{kb/pkt}}{10^{**}9 \text{ b/sec}} = 8 \text{ microsec}$$

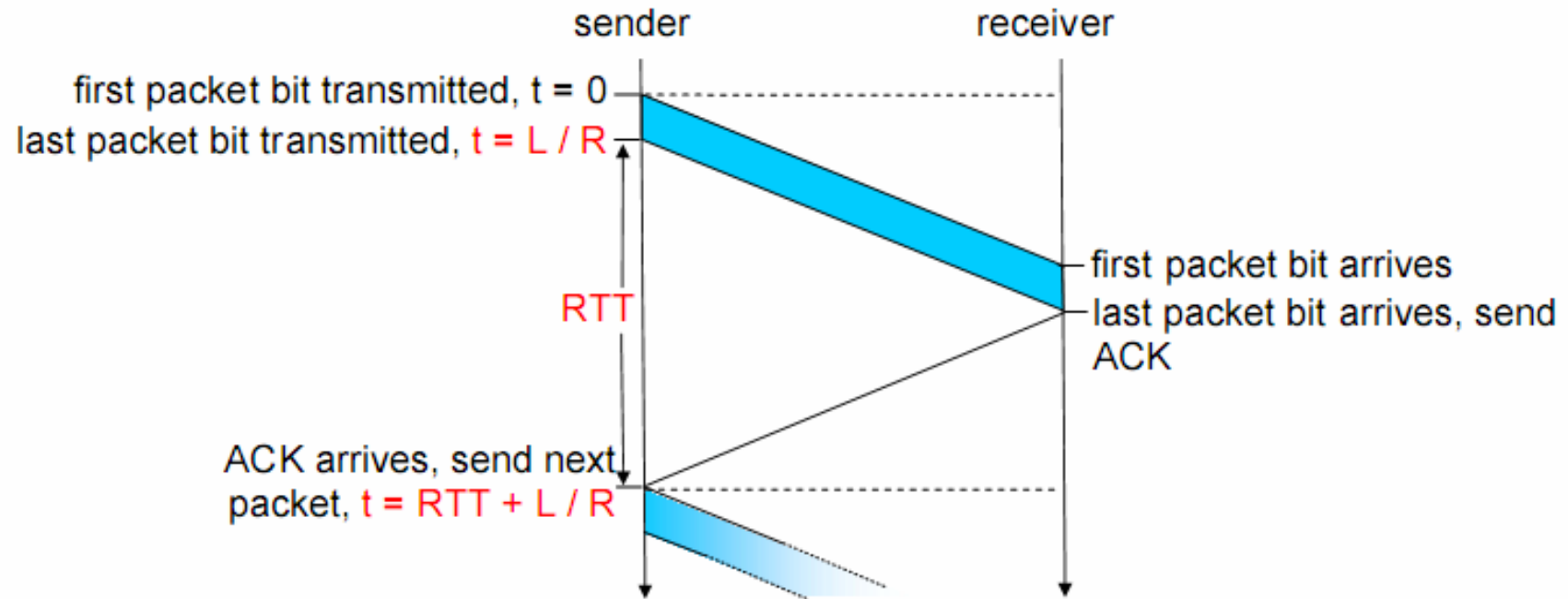
tfd3.0 – Características y Rendimiento

- Utilización del canal del emisor
- Fracción de tiempo en que el emisor esta realmente ocupado emitiendo bits en el canal.

$$U_{\text{sender}} = \frac{L / R}{RTT + L / R} = \frac{.008}{30.008} = 0.00027$$

- El emisor solo utiliza 2,7 centésimas partes del uno por ciento del tiempo disponible
- Solo puede emitir 1000 bytes en 30.008 ms. ($1000 * 8 / 30.008$)
- Productividad efectiva de 267 kbps sobre un enlace de 1Gb!!
- Protocolo LIMITA los recursos físicos de la red

tfd3.0 – operación de parada y espera



$$U_{\text{sender}} = \frac{L / R}{RTT + L / R} = \frac{.008}{30.008} = 0.00027$$

Stop and Wait Link Utilization

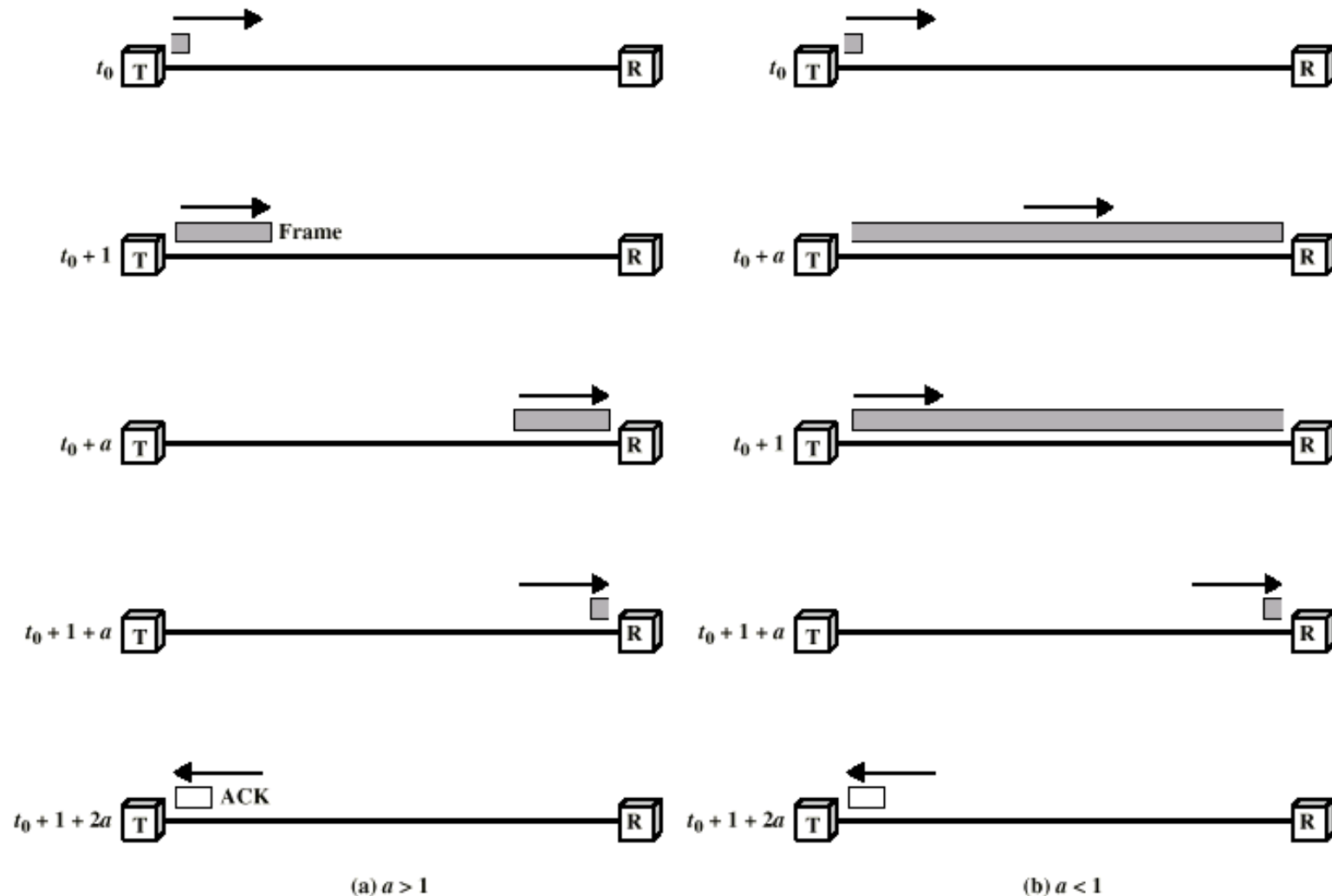


Figure 7.2 Stop-and-Wait Link Utilization (transmission time = 1; propagation time = a)

Error Detection

- Additional bits added by transmitter for error detection code
- Parity
 - Value of parity bit is such that character has even (even parity) or odd (odd parity) number of ones
 - Even number of bit errors goes undetected

Cyclic Redundancy Check

- For a block of k bits transmitter generates n bit sequence
- Transmit $k+n$ bits which is exactly divisible by some number
- Receive divides frame by that number
 - If no remainder, assume no error
 - For math, see Stallings chapter 7

Error Control

- Detection and correction of errors
- Lost frames
- Damaged frames
- Automatic repeat request
 - Error detection
 - Positive acknowledgment
 - Retransmission after timeout
 - Negative acknowledgement and retransmission

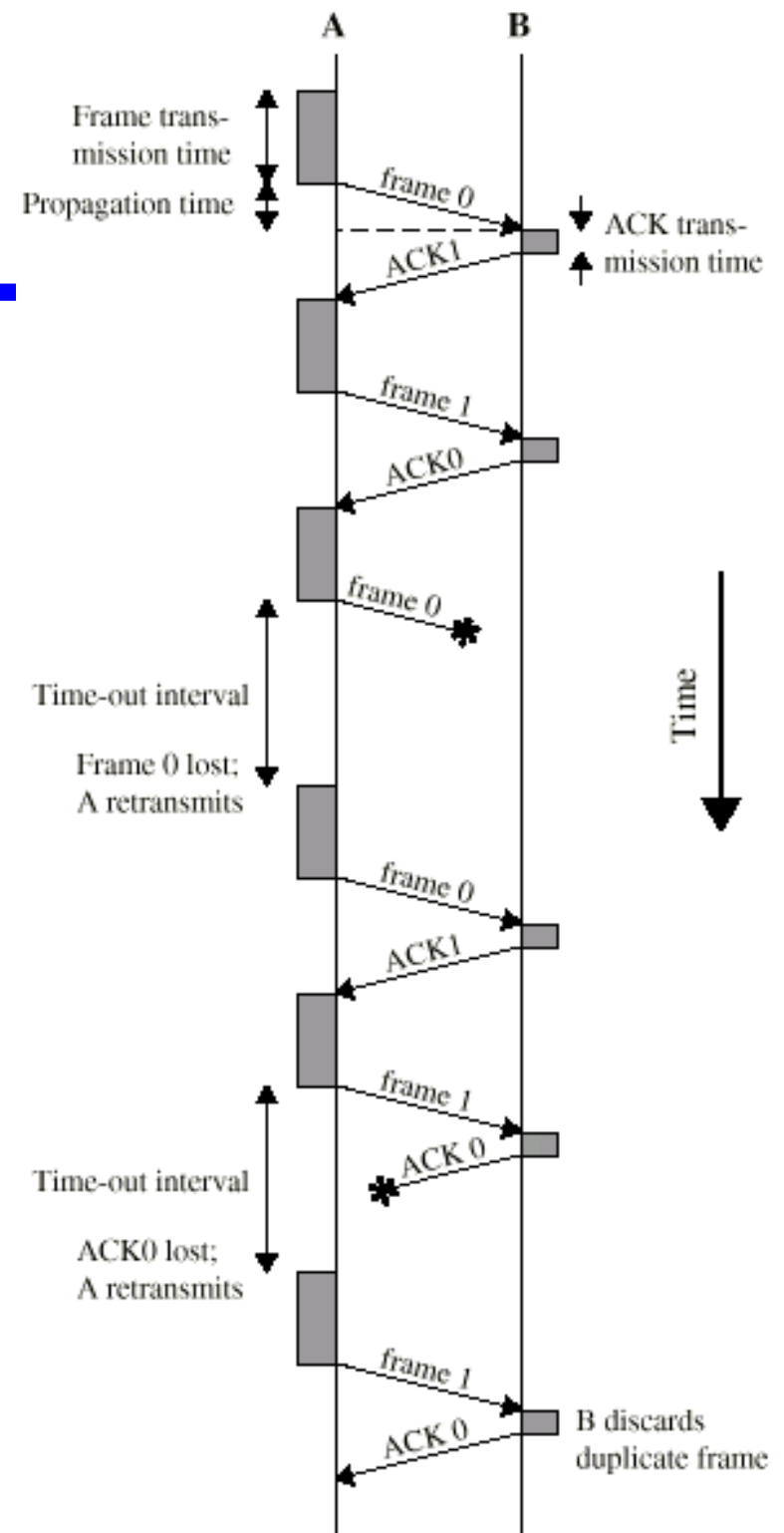
Automatic Repeat Request (ARQ)

- Stop and wait
- Go back N
- Selective reject (selective retransmission)

Stop and Wait

- Source transmits single frame
- Wait for ACK
- If received frame damaged, discard it
 - Transmitter has timeout
 - If no ACK within timeout, retransmit
- If ACK damaged, transmitter will not recognize it
 - Transmitter will retransmit
 - Receiver gets two copies of frame
 - Use ACK0 and ACK1

Stop and Wait - Diagram



Stop and Wait - Pros and Cons

- Simple
- Inefficient